NumPy for Teaching: A Complete Guide

1. What is NumPy?

NumPy (Numerical Python) is the core library for numerical and scientific computing in Python. It provides a powerful data structure called the **ndarray** (n-dimensional array), along with a huge collection of mathematical functions to operate on these arrays efficiently.

Almost every data science, machine learning, and scientific computing library in Python (Pandas, Scikit-learn, TensorFlow, PyTorch, SciPy) is built on top of NumPy.

Why was NumPy created?

Python's built-in `list` is flexible but slow for numerical computation. NumPy was built to solve this by:

- Storing data in a compact, fixed-type, contiguous block of memory
- Performing operations in optimized, pre-compiled C code
- Supporting vectorized operations (no explicit loops needed)

Installing NumPy

```
pip install numpy
```

Importing NumPy

```
import numpy as np
```

`np` is the universally used alias for NumPy — always use it so your code is consistent with the rest of the community.

---

2. What is an Array?

An **array** is a data structure that stores a collection of values, all of the **same data type**, arranged in a grid. Arrays can be:

- **1D (vector):** a single row of values → `[1, 2, 3, 4]`
- **2D (matrix):** rows and columns → `[[1, 2], [3, 4]]`
- **3D or higher (tensor):** stacked matrices, used heavily in deep learning

In NumPy, this structure is implemented as the `ndarray` object.

Creating your first array

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)          # [1 2 3 4 5]
print(type(arr))    # <class 'numpy.ndarray'>
```

Creating a 2D array

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print(matrix)
# [[1 2 3]
#  [4 5 6]]
print(matrix.shape)  # (2, 3) -> 2 rows, 3 columns
```

Useful array-creation functions

```
np.zeros((2, 3))       # array of all zeros, shape (2,3)
np.ones((3, 3))        # array of all ones
np.full((2, 2), 7)     # array filled with the value 7
np.eye(3)              # 3x3 identity matrix
np.arange(0, 10, 2)    # [0 2 4 6 8]  (like Python's range)
np.linspace(0, 1, 5)   # [0. 0.25 0.5 0.75 1.]  5 evenly spaced values
np.random.rand(2, 2)   # 2x2 array of random floats between 0 and 1
```

---

3. Array vs List — The Core Topic

This is one of the most important concepts to understand before using NumPy effectively.

Feature	Python List	NumPy Array
Data type	Can hold mixed types (`[1, "a", 3.5]`)	Must hold a single data type (all int, all float, etc.)
Memory usage	Higher — stores pointers to objects scattered in memory	Lower — stores data in one contiguous block
Speed	Slower for numeric operations (uses Python-level loops)	Much faster (uses optimized C loops, vectorized operations)
Mathematical operations	Not directly supported (`[1,2] + [3,4]` → concatenation, not addition)	Fully supported (`+`, `-`, `*`, `/` work element-wise)
Size flexibility	Dynamic — can grow/shrink easily (`append`, `remove`)	Fixed size once created; resizing creates a new array
Dimensions	Only naturally 1D (nested lists simulate 2D/3D but are clunky)	Native support for 1D, 2D, 3D, and n-D arrays
Built-in functions	Basic list operations only	Huge library: linear algebra, statistics, broadcasting, FFT, etc.
Import needed	Built into Python	Requires `import numpy`

Example 1: Mixed types

```
# Python list — allows mixed types
my_list = [1, "hello", 3.5, True]
```

```
print(my_list)   # [1, 'hello', 3.5, True]

# NumPy array — forces a single type (upcasts to string here)
import numpy as np
my_array = np.array([1, "hello", 3.5, True])
print(my_array)         # ['1' 'hello' '3.5' 'True']
print(my_array.dtype)   # dtype('<U32') -> all converted to strings
```

Example 2: Mathematical operations

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

# Lists: + means concatenation, not addition
print(list1 + list2)          # [1, 2, 3, 4, 5, 6]

# You'd need a loop or list comprehension to add element-wise
result = [a + b for a, b in zip(list1, list2)]
print(result)                 # [5, 7, 9]

# NumPy arrays: + performs element-wise addition automatically
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print(arr1 + arr2)            # [5 7 9]
print(arr1 * arr2)            # [4 10 18]  (element-wise
multiplication)
print(arr1 ** 2)              # [1 4 9]    (element-wise power)
```

Example 3: Speed difference

```
import numpy as np
import time

size = 1_000_000
list1 = list(range(size))
list2 = list(range(size))

start = time.time()
result = [a + b for a, b in zip(list1, list2)]
print("List time:", time.time() - start)

arr1 = np.arange(size)
arr2 = np.arange(size)

start = time.time()
result = arr1 + arr2
print("NumPy time:", time.time() - start)
```

NumPy will typically be **10–50x faster** or more for large datasets, because the addition happens in compiled C code instead of a Python-level loop.

Example 4: Multi-dimensional handling

```
# Simulating a 2D structure with lists (nested lists)
list_2d = [[1, 2, 3], [4, 5, 6]]
print(list_2d[1][2])   # 6  -> manual indexing, no built-in matrix operations

# NumPy handles this natively and efficiently
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr_2d[1, 2])     # 6  -> cleaner indexing
print(arr_2d.sum())     # 21 -> built-in aggregate functions
print(arr_2d.T)         # transpose the matrix instantly
```

Summary — when to use which

- Use a **list** when you need a simple, flexible, general-purpose container with mixed data types, or when doing lightweight, non-numeric tasks.
- Use a **NumPy array** when doing numerical computation, especially with large datasets, matrices, or when speed and memory efficiency matter.

---

4. Array Attributes

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.ndim)     # 2  -> number of dimensions
print(arr.shape)    # (2, 3) -> rows, columns
print(arr.size)     # 6  -> total number of elements
print(arr.dtype)    # int64 -> data type of elements
print(arr.itemsize) # 8  -> bytes per element
```

---

5. Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])       # 10           -> first element
print(arr[-1])      # 50           -> last element
print(arr[1:4])     # [20 30 40]   -> slice
print(arr[::-1])    # [50 40 30 20 10] -> reversed

matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[0, :])   # [1 2 3]    -> first row
print(matrix[:, 1])   # [2 5 8]    -> second column
print(matrix[1:, 1:]) # [[5 6] [8 9]] -> sub-matrix
```

Boolean (fancy) indexing

```
arr = np.array([1, 2, 3, 4, 5, 6])
print(arr[arr > 3])       # [4 5 6]
arr[arr % 2 == 0] = 0     # replace all even numbers with 0
print(arr)                # [1 0 3 0 5 0]
```

---

6. Reshaping Arrays

```
arr = np.arange(1, 13)      # [1, 2, ..., 12]
print(arr.reshape(3, 4))
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

flat = arr.reshape(3, 4).flatten()
print(flat)  # back to 1D
```

---

7. Broadcasting

Broadcasting lets NumPy perform operations on arrays of different shapes without explicit loops.

```
arr = np.array([1, 2, 3])
print(arr + 10)          # [11 12 13]  -> scalar "broadcast" to every
element

matrix = np.array([[1, 2, 3], [4, 5, 6]])
row = np.array([10, 20, 30])
print(matrix + row)
# [[11 22 33]
#  [14 25 36]]
```

---

8. Statistical Methods in NumPy — What They Are and When to Use Them

NumPy provides a full set of statistical functions that work directly on arrays. This section explains what each method calculates, and — just as importantly — **when a teacher or student should reach for it**.

8.1 Measures of Central Tendency (the "typical" value)

Mean — `np.mean()`

What it is: the arithmetic average — sum of all values divided by the count.

When to use it: use the mean when your data is fairly evenly distributed with no extreme outliers (e.g., average test score of a class with consistent performance). Avoid it when a few extreme values could skew the result.

```
marks = np.array([56, 78, 90, 45, 67, 89])
print(np.mean(marks))   # 70.83...
```

Median — `np.median()`

What it is: the middle value when data is sorted (or the average of the two middle values for an even count).

When to use it: use the median when your data has outliers or is skewed — for example, household income or house prices, where a few very high values would distort the mean.

```
salaries = np.array([25000, 27000, 26000, 500000, 24000])
print(np.mean(salaries))    # heavily pulled up by the outlier
print(np.median(salaries))  # gives a more "typical" picture
```

Mode — via `scipy.stats.mode()` (NumPy has no direct built-in, but it's commonly taught alongside it)

What it is: the most frequently occurring value.

When to use it: use the mode for categorical or discrete data — e.g., the most common shoe size sold, or the most frequent grade in a class.

```
from scipy import stats
grades = np.array([70, 80, 80, 90, 60, 80])
print(stats.mode(grades, keepdims=True))  # mode = 80
```

8.2 Measures of Spread (how "spread out" the data is)

Variance — `np.var()`

What it is: the average of the squared differences from the mean — tells you how much the data varies overall.

When to use it: use it when you want a mathematical measure of spread for further calculations (variance is the building block for standard deviation and many statistical models). It's harder to interpret directly because it's in squared units.

```
data = np.array([4, 8, 6, 5, 3])
print(np.var(data))   # variance
```

Standard Deviation — `np.std()`

What it is: the square root of variance — spread measured in the *same units* as the original data, which makes it easier to interpret.

When to use it: use it when you want to describe how consistent or inconsistent data is — e.g., "test scores had a low standard deviation" means most students scored close to the average. Widely used in quality control, finance, and grading.

```
print(np.std(data))    # standard deviation
```

Range — `np.max() - np.min()`

What it is: the difference between the highest and lowest values.

When to use it: use it for a quick, simple sense of spread, especially with small datasets — but it's sensitive to outliers since it only looks at the two extremes.

```
print(np.max(data) - np.min(data))
```

8.3 Position-Based Statistics

Percentile — `np.percentile()`

What it is: the value below which a given percentage of the data falls. For example, the 90th percentile is the value below which 90% of the data lies.

When to use it: use it for ranking and comparison — e.g., "your child scored in the 95th percentile" tells you their score relative to everyone else, which is more informative than the raw score alone. Common in standardized testing, growth charts, and performance benchmarking.

```
scores = np.array([55, 60, 65, 70, 75, 80, 85, 90, 95, 100])
print(np.percentile(scores, 90))  # value at the 90th percentile
```

Quantiles / Quartiles — `np.percentile()` at 25, 50, 75

What it is: percentiles at fixed intervals — the 25th (Q1), 50th (Q2/median), and 75th (Q3) percentiles split data into four equal parts.

When to use it: use quartiles to describe the spread of the middle 50% of your data (the interquartile range, or IQR), which is useful for detecting outliers robustly.

```
q1 = np.percentile(scores, 25)
q3 = np.percentile(scores, 75)
iqr = q3 - q1
print("IQR:", iqr)
```

8.4 Relationship Between Variables

Correlation — `np.corrcoef()`

What it is: measures how strongly two variables move together, on a scale from -1 (perfectly opposite) to +1 (perfectly together); 0 means no linear relationship.

When to use it: use it when you want to check if two variables are related — e.g., "does study time correlate with exam score?" Useful before building predictive models, to spot which variables matter.

```
study_hours = np.array([1, 2, 3, 4, 5])
exam_scores = np.array([50, 55, 65, 70, 85])
print(np.corrcoef(study_hours, exam_scores))
```

8.5 Cumulative and Aggregate Functions

Sum — `np.sum()`

What it is: total of all values.

When to use it: whenever you need a total — total sales, total marks, total items.

Cumulative Sum — `np.cumsum()`

What it is: a running total at each position in the array.

When to use it: useful for tracking totals over time — e.g., cumulative revenue by month, or running total of steps walked in a week.

```
daily_sales = np.array([100, 150, 200, 130])
print(np.cumsum(daily_sales))  # [100 250 450 580]
```

Cumulative Product — `np.cumprod()`

What it is: a running product at each position.

When to use it: useful for compounding calculations — e.g., compound interest or growth rates over time.

8.6 Quick Reference — Which Statistic to Use When

Question you're asking	Use this
"What's the typical/average value?"	`np.mean()` (no major outliers) or `np.median()` (skewed data / outliers)
"What value appears most often?"	Mode (via `scipy.stats.mode`)
"How spread out is my data?"	`np.std()` for interpretable spread, `np.var()` for raw variance
"What's the full range of values?"	`np.max() - np.min()`
"How does one value rank compared to everyone else?"	`np.percentile()`
"What's the spread of the middle 50% of data?"	IQR using 25th/75th percentiles
"Are two variables related?"	`np.corrcoef()`
"What's the running total over time?"	`np.cumsum()`
"What's the compounding growth over time?"	`np.cumprod()`

8.7 Worked Example — Putting Statistics Together

```
import numpy as np

marks = np.array([56, 78, 90, 45, 67, 89, 34, 95, 72, 60])

print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Std Dev:", np.std(marks))
print("Variance:", np.var(marks))
print("Min/Max:", np.min(marks), np.max(marks))
print("25th percentile:", np.percentile(marks, 25))
print("75th percentile:", np.percentile(marks, 75))
print("IQR:", np.percentile(marks, 75) - np.percentile(marks, 25))
```

This kind of exercise is great for teaching students to move beyond just "the average" and think about spread, ranking, and outliers when describing a dataset.

---

9. A Small Classroom Exercise

Task: Given a list of student marks, calculate who scored above average using both a plain list and a NumPy array, to compare the code.

```
marks_list = [56, 78, 90, 45, 67, 89]

# Using a list
avg = sum(marks_list) / len(marks_list)
above_avg = [m for m in marks_list if m > avg]
print(above_avg)

# Using NumPy
marks_arr = np.array(marks_list)
avg = marks_arr.mean()
above_avg = marks_arr[marks_arr > avg]
print(above_avg)
```

This side-by-side comparison is a great way to show students how NumPy simplifies numeric logic that would otherwise need loops and conditionals.

---

10. Key Takeaways for Students

1. NumPy's core object is the **ndarray** — a fast, memory-efficient, same-type grid of values.
2. Lists are general-purpose and flexible; arrays are specialized for numeric computation and are much faster.
3. Arithmetic operators behave differently: lists concatenate, arrays operate element-wise.
4. NumPy supports multi-dimensional data naturally, along with reshaping, slicing, and broadcasting.
5. Statistical methods aren't interchangeable — mean vs. median, variance vs. standard deviation, and percentiles all answer different questions, and picking the right one matters for accurate interpretation.
6. NumPy is the foundation for almost every data science and machine learning library in Python.